

Universidad Panamericana – Ciudad UP

Notebook documentado en Markdown y $\text{\LaTeX}$

Series de Tiempo: ARIMA, SARIMA, SARIMAX y GARCH

Elaborado para: Sofia Escamilla

Curso: Series de Tiempo / Finanzas Cuantitativas

Este documento convierte los materiales de Series de Tiempo en una guía tipo Google Colab, con celdas de Markdown, fórmulas en $\text{\LaTeX}$, código reproducible en Python, metodología, diagnóstico e interpretación.

Índice

1. Alcance del notebook y separación correcta de materiales	1
1.1. Archivos considerados	1
1.2. Objetivo general	1
1.3. Estructura recomendada en Colab	1
2. Preparación del entorno en Google Colab	2
2.1. Celda 1: instalación de dependencias	2
2.2. Celda 2: importación de librerías	2
2.3. Celda 3: carga de documentos a Colab	3
3. Carga, limpieza y preparación de la base de ventas	3
3.1. Descripción de la base	3
3.2. Celda 4: lectura del archivo CSV	3
3.3. Celda 5: revisión inicial	4
3.4. Celda 6: transformación de fechas y ordenamiento	4
3.5. Celda 7: construcción de una serie diaria agregada	4
3.6. Celda 8: tratamiento de datos faltantes	4
4. Análisis exploratorio de la serie	5
4.1. Celda 9: gráfica de ventas agregadas	5
4.2. Celda 10: descomposición por día de la semana	5
4.3. Celda 11: análisis mensual	5
5. Marco teórico: ARIMA, SARIMA y SARIMAX	6
5.1. Serie de tiempo y estacionariedad	6
5.2. Modelo ARIMA	6
5.3. Modelo SARIMA	7
5.4. Modelo SARIMAX	7
6. Estacionariedad y transformaciones	8
6.1. Celda 12: prueba ADF	8
6.2. Celda 13: transformación logarítmica y diferenciación	8
7. Identificación con ACF y PACF	9
7.1. Fundamento teórico	9

7.2. Celda 14: correlogramas	9
8. Estimación de modelos ARIMA y SARIMA	9
8.1. Criterios de selección	10
8.2. Celda 15: función para ajustar SARIMA	10
8.3. Celda 16: modelo ARIMA base	10
8.4. Celda 17: modelo SARIMA con estacionalidad semanal	10
8.5. Celda 18: comparación de modelos	11
9. Modelo SARIMAX con variables exógenas	11
9.1. Motivación	11
9.2. Celda 19: preparación de variables exógenas	11
9.3. Celda 20: comparación final	11
10. Diagnóstico de residuos	12
10.1. Prueba Ljung–Box	12
10.2. Celda 21: diagnóstico de residuos	12
10.3. Celda 22: normalidad y distribución de residuos	12
11. Pronóstico con intervalos de confianza	13
11.1. Celda 23: pronóstico fuera de muestra	13
11.2. Celda 24: gráfica de pronóstico	13
12. Apéndice teórico: ejercicios SARIMAX y GARCH	14
12.1. SARIMAX con variable exógena	14
12.2. Prueba ARCH	14
12.3. Modelo GARCH(1,1)	15
12.4. GJR-GARCH y efecto apalancamiento	15
12.5. EGARCH	15
12.6. GARCH-M	16
13. Celda opcional: estimación GARCH en Python	16
14. Conclusiones del notebook	17
15. Checklist final para ejecutar en Colab	17

Índice de cuadros

1.	Archivos de Series de Tiempo integrados al notebook	1
2.	Parámetros principales del modelo ARIMA	7
3.	Parámetros del modelo SARIMA	7
4.	Patrones orientativos de ACF y PACF	9
5.	Checklist de revisión del notebook	17

1 Alcance del notebook y separación correcta de materiales

Celda Markdown

Este notebook integra únicamente materiales de **Series de Tiempo**. No se incorporan documentos de Cálculo Estocástico, valuación de opciones, Movimiento Browniano Geométrico, Monte Carlo, Longstaff–Schwartz, opciones americanas ni opciones parisinas. La finalidad es mantener una entrega limpia y coherente con el curso de Series de Tiempo.

1.1 Archivos considerados

Cuadro 1: Archivos de Series de Tiempo integrados al notebook

Archivo	Uso dentro del notebook
store_sales (1).csv	Base de datos aplicada para análisis de ventas por tienda, con variables de fecha, tienda, ventas, promoción y feriado.
README.md	Estructura conceptual del repositorio: introducción, ARIMA/SARIMA, diagnóstico, selección de modelos y volatilidad financiera.
Reporte Sarima (1).pdf y Reporte Sarima (2).pdf	Material teórico sobre ARIMA, SARIMA, SARIMAX, ACF, PACF, parsimonia, AIC, BIC y prueba Ljung–Box.
Proyecto ST.Escamilla, Guevara(1).tex	Fuente con ejercicios analíticos: SARIMAX, prueba ARCH, GARCH, GJR-GARCH, EGARCH, GARCH-M, VaR y validación ARIMA-GARCH.
Proyecto Series. Escamilla, Guevara (1)(1).pdf	Versión compilada del proyecto de ejercicios y soluciones de Series de Tiempo.
informe_arima_sarima_aapl (1).tex	Referencia para estructurar un informe aplicado con ARIMA, SARIMA, ACF, PACF, AIC, BIC y diagnóstico de residuos.

1.2 Objetivo general

Celda Markdown

El objetivo es construir un notebook reproducible en Google Colab que permita cargar una base de datos de ventas, preparar una serie temporal, diagnosticar estacionariedad, identificar patrones con ACF y PACF, estimar modelos ARIMA/SARIMA/SARIMAX, validar residuos, comparar modelos con criterios de información y, de forma complementaria, documentar modelos de volatilidad financiera ARCH/GARCH.

1.3 Estructura recomendada en Colab

1. Cargar librerías y archivos del proyecto.
2. Limpiar y preparar la base store_sales.csv.

3. Construir la serie temporal agregada o por tienda.
4. Realizar análisis exploratorio.
5. Evaluar estacionariedad con ADF y transformaciones.
6. Identificar parámetros con ACF y PACF.
7. Estimar ARIMA, SARIMA y SARIMAX.
8. Comparar AIC, BIC, log-verosimilitud y errores.
9. Validar residuos con Ljung–Box y análisis visual.
10. Generar pronósticos e interpretar resultados.

2 Preparación del entorno en Google Colab

2.1 Celda 1: instalación de dependencias

Celda Markdown

En Google Colab se recomienda instalar explícitamente las librerías necesarias para que el análisis sea reproducible. Aunque varias ya vienen instaladas, se deja la celda completa para evitar errores al ejecutar el notebook desde cero.

```
!pip install pandas numpy matplotlib seaborn scipy statsmodels pmdarima arch
--quiet
```

2.2 Celda 2: importación de librerías

```
1 import warnings
2 warnings.filterwarnings("ignore")
3
4 from pathlib import Path
5 import numpy as np
6 import pandas as pd
7 import matplotlib.pyplot as plt
8
9 from scipy import stats
10 from statsmodels.tsa.stattools import adfuller
11 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
12 from statsmodels.tsa.statespace.sarimax import SARIMAX
13 from statsmodels.stats.diagnostic import acorr_ljungbox, het_arch
14
15 try:
16     from arch import arch_model
17 except ImportError:
18     arch_model = None
19
20 plt.rcParams["figure.figsize"] = (12, 5)
21 plt.rcParams["axes.grid"] = True
```

2.3 Celda 3: carga de documentos a Colab

Celda Markdown

Para “jalar” los archivos al entorno de Colab se puede usar carga manual o Google Drive. La opción manual funciona cuando se suben los archivos directamente desde la computadora.

```
1 from google.colab import files
2
3 uploaded = files.upload()
4 for file_name in uploaded.keys():
5     print(f"Archivo cargado: {file_name}")
```

Celda Markdown

Si el proyecto se guarda en Google Drive, conviene montar la unidad y definir una ruta fija del proyecto.

```
1 from google.colab import drive
2 drive.mount('/content/drive')
3
4 project_path = Path('/content/drive/MyDrive/series_de_tiempo')
5 project_path.mkdir(parents=True, exist_ok=True)
6 list(project_path.iterdir())
```

3 Carga, limpieza y preparación de la base de ventas

3.1 Descripción de la base

Celda Markdown

La base `store_sales (1).csv` contiene una serie diaria de ventas. Sus columnas principales son:

- `date`: fecha de observación.
- `store`: identificador de tienda.
- `sales`: ventas observadas.
- `promo`: variable binaria que indica si hubo promoción.
- `holiday`: variable binaria que indica si el día fue feriado.

Estas variables permiten construir modelos de series de tiempo con y sin variables exógenas. En particular, `promo` y `holiday` pueden incorporarse como regresores externos en un modelo SARIMAX.

3.2 Celda 4: lectura del archivo CSV

```
1 # Ruta local cuando el archivo se carga directamente a Colab
2 csv_path = Path('store_sales (1).csv')
3
4 # Alternativa si se usa Google Drive:
5 # csv_path = project_path / 'store_sales (1).csv'
6
7 df = pd.read_csv(csv_path)
8 df.head()
```

3.3 Celda 5: revisión inicial

```
1 print(df.shape)
2 print(df.dtypes)
3 print(df.isna().sum())
4 df.describe(include='all')
```

3.4 Celda 6: transformación de fechas y ordenamiento

```
1 df['date'] = pd.to_datetime(df['date'])
2 df = df.sort_values(['store', 'date']).reset_index(drop=True)
3
4 print(df['date'].min(), df['date'].max())
5 print('Número de tiendas:', df['store'].nunique())
```

3.5 Celda 7: construcción de una serie diaria agregada

Celda Markdown

Para un primer modelo de series de tiempo se agregan las ventas de todas las tiendas por fecha. También se promedian las variables binarias `promo` y `holiday`; esto permite medir la proporción de tiendas con promoción o feriado en cada día.

```
1 daily = (
2     df.groupby('date')
3     .agg(
4         sales=('sales', 'sum'),
5         promo=('promo', 'mean'),
6         holiday=('holiday', 'mean')
7     )
8     .asfreq('D')
9 )
10
11 daily.head()
```

3.6 Celda 8: tratamiento de datos faltantes


```
1 print(daily.isna().sum())
2
3 # Si existieran fechas sin observación, se imputan de manera conservadora.
4 daily['sales'] = daily['sales'].interpolate(method='time')
5 daily['promo'] = daily['promo'].fillna(0)
6 daily['holiday'] = daily['holiday'].fillna(0)
```

Interpretación financiera / econométrica

La interpolación se usa únicamente si faltan fechas intermedias. En una serie de ventas, eliminar fechas puede distorsionar la estacionalidad semanal o mensual. Por eso conviene mantener una frecuencia diaria constante.

4 Análisis exploratorio de la serie

4.1 Celda 9: gráfica de ventas agregadas

```
1 fig, ax = plt.subplots()
2 daily['sales'].plot(ax=ax)
3 ax.set_title('Ventas diarias agregadas')
4 ax.set_xlabel('Fecha')
5 ax.set_ylabel('Ventas')
6 plt.show()
```

4.2 Celda 10: descomposición por día de la semana

```
1 daily['day_of_week'] = daily.index.day_name()
2 weekly_pattern = daily.groupby('day_of_week')['sales'].mean()
3 weekly_pattern = weekly_pattern.reindex([
4     'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'
5 ])
6
7 weekly_pattern.plot(kind='bar')
8 plt.title('Ventas promedio por día de la semana')
9 plt.ylabel('Ventas promedio')
10 plt.show()
```

4.3 Celda 11: análisis mensual

```
1 monthly_sales = daily['sales'].resample('M').sum()
2 monthly_sales.plot(marker='o')
3 plt.title('Ventas mensuales agregadas')
4 plt.ylabel('Ventas mensuales')
5 plt.show()
```

Interpretación financiera / econométrica

La visualización inicial permite detectar tendencia, estacionalidad y cambios de nivel. En series de ventas es común encontrar estacionalidad semanal, efectos de calendario, impacto de promociones y posibles picos por feriados.

5 Marco teórico: ARIMA, SARIMA y SARIMAX

5.1 Serie de tiempo y estacionariedad

Celda Markdown

Una serie de tiempo es una secuencia de observaciones ordenadas en el tiempo. En econometría y finanzas, el orden temporal importa porque una observación puede depender de su propio pasado.

Celda LaTeX / Fórmula

Una serie $\{y_t\}$ es débilmente estacionaria si cumple:

$$E(y_t) = \mu, \quad Var(y_t) = \sigma^2, \quad Cov(y_t, y_{t-k}) = \gamma_k.$$

Es decir, su media y varianza no dependen del tiempo, y la autocovarianza depende sólo del rezago k .

5.2 Modelo ARIMA

Celda Markdown

Un modelo ARIMA combina tres componentes: autorregresivo, integración y media móvil. Se denota como $ARIMA(p, d, q)$, donde p es el orden AR, d es el número de diferencias necesarias para estacionarizar la serie y q es el orden MA.

Celda LaTeX / Fórmula

Usando el operador rezago B , donde $B^k y_t = y_{t-k}$, el modelo ARIMA puede escribirse como:

$$\phi_p(B)(1 - B)^d y_t = \theta_0 + \theta_q(B)\varepsilon_t, \quad \varepsilon_t \sim WN(0, \sigma^2).$$

Los polinomios son:

$$\phi_p(B) = 1 - \phi_1 B - \dots - \phi_p B^p, \quad \theta_q(B) = 1 + \theta_1 B + \dots + \theta_q B^q.$$

Cuadro 2: Parámetros principales del modelo ARIMA

Parámetro	Nombre	Interpretación
p	Orden AR	Número de rezagos autorregresivos.
d	Integración	Número de diferencias regulares.
q	Orden MA	Número de rezagos del error.

5.3 Modelo SARIMA

Celda Markdown

El modelo SARIMA extiende ARIMA al incorporar componentes estacionales. Es útil cuando la serie presenta ciclos repetidos, por ejemplo semanalidad en datos diarios, estacionalidad mensual con periodo $s = 12$, o estacionalidad trimestral con $s = 4$.

Celda LaTeX / Fórmula

La notación completa es:

$$SARIMA(p, d, q)(P, D, Q)_s.$$

Su forma general es:

$$\Phi_P(B^s)\phi_p(B)(1 - B^s)^D(1 - B)^d y_t = \Theta_Q(B^s)\theta_q(B)\varepsilon_t.$$

Cuadro 3: Parámetros del modelo SARIMA

Parámetro	Componente	Descripción
p	AR regular	Rezagos autorregresivos no estacionales.
d	Diferencia regular	Diferencias para remover tendencia.
q	MA regular	Rezagos de media móvil no estacionales.
P	AR estacional	Rezagos autorregresivos estacionales.
D	Diferencia estacional	Diferencias para remover estacionalidad integrada.
Q	MA estacional	Rezagos de media móvil estacionales.
s	Periodo	Longitud del ciclo estacional.

5.4 Modelo SARIMAX

Celda Markdown

SARIMAX agrega variables exógenas al modelo SARIMA. Esto permite explicar la serie no sólo con su propia dinámica histórica, sino también con factores externos, por ejemplo promociones, feriados, indicadores macroeconómicos o precios de materias primas.

Celda LaTeX / Fórmula

Si $X_t = (x_{1t}, x_{2t}, \dots, x_{rt})'$ es el vector de variables exógenas, entonces:

$$\Phi_P(B^s)\phi_p(B)(1-B^s)^D(1-B)^d y_t = \beta' X_t + \Theta_Q(B^s)\theta_q(B)\varepsilon_t.$$

Interpretación financiera / econométrica

En la base de ventas, un SARIMAX puede incorporar **promo** y **holiday**. Si el coeficiente de **promo** es positivo y significativo, la interpretación sería que los días con promoción tienden a asociarse con mayores ventas, controlando por la dinámica temporal de la serie.

6 Estacionariedad y transformaciones

6.1 Celda 12: prueba ADF

Celda Markdown

La prueba ADF contrasta la hipótesis nula de raíz unitaria. Si no se rechaza la hipótesis nula, la serie no es estacionaria y puede requerir diferenciación.

Celda LaTeX / Fórmula

H_0 : la serie tiene raíz unitaria, H_1 : la serie es estacionaria.

```
1 def adf_test(series, name='serie'):
2     series = series.dropna()
3     result = adfuller(series, autolag='AIC')
4     print(f'Prueba ADF para {name}')
5     print(f'Estadístico ADF: {result[0]:.4f}')
6     print(f'p-value: {result[1]:.4f}')
7     print('Valores críticos:')
8     for key, value in result[4].items():
9         print(f' {key}: {value:.4f}')
10
11 adf_test(daily['sales'], 'ventas diarias')
```

6.2 Celda 13: transformación logarítmica y diferenciación

```
1 daily['log_sales'] = np.log(daily['sales'])
2 daily['diff_log_sales'] = daily['log_sales'].diff()
3
4 adf_test(daily['log_sales'], 'log ventas')
5 adf_test(daily['diff_log_sales'], 'diferencia log ventas')
```

Interpretación financiera / econométrica

La transformación logarítmica ayuda a estabilizar la varianza. La diferencia logarítmica se interpreta aproximadamente como una tasa de crecimiento continua. Si la serie original no es estacionaria pero la diferencia logarítmica sí lo es, entonces un modelo con $d = 1$ puede ser adecuado.

7 Identificación con ACF y PACF

7.1 Fundamento teórico

Celda Markdown

La ACF mide autocorrelaciones totales entre y_t y y_{t-k} . La PACF mide autocorrelaciones parciales, controlando por los rezagos intermedios. En la identificación Box–Jenkins, ambas funciones ayudan a proponer valores iniciales de p y q .

Cuadro 4: Patrones orientativos de ACF y PACF

Proceso	ACF	PACF
AR(p)	Decaimiento gradual	Corte después del rezago p .
MA(q)	Corte después del rezago q	Decaimiento gradual.
ARMA(p, q)	Decaimiento gradual	Decaimiento gradual.
Ruido blanco	Sin autocorrelación significativa	Sin autocorrelación significativa.

7.2 Celda 14: correlogramas

```

1 serie_modelo = daily['diff_log_sales'].dropna()
2
3 fig, ax = plt.subplots(1, 2, figsize=(14, 4))
4 plot_acf(serie_modelo, lags=40, ax=ax[0])
5 plot_pacf(serie_modelo, lags=40, ax=ax[1], method='ywm')
6 ax[0].set_title('ACF de la serie transformada')
7 ax[1].set_title('PACF de la serie transformada')
8 plt.show()

```

Cuidado metodológico

La ACF y la PACF no deben usarse de manera mecánica. Funcionan como guía inicial, pero la decisión final debe considerar parsimonia, significancia, criterios AIC/BIC, diagnóstico de residuos y sentido económico.

8 Estimación de modelos ARIMA y SARIMA

8.1 Criterios de selección

Celda LaTeX / Fórmula

Los criterios de información más usados son:

$$AIC = -2\ln(\hat{L}) + 2k, \quad BIC = -2\ln(\hat{L}) + k\ln(n),$$

donde $\hat{L}$ es la verosimilitud maximizada, k es el número de parámetros y n el tamaño de muestra. En ambos casos, se prefiere el valor más bajo.

Interpretación financiera / econométrica

El BIC penaliza más la complejidad cuando n es grande, por lo que suele elegir modelos más conservadores. Esto se relaciona con el principio de parsimonia: no usar más parámetros de los necesarios.

8.2 Celda 15: función para ajustar SARIMA

```

1 def fit_sarima(y, order, seasonal_order=(0,0,0,0), exog=None):
2     model = SARIMAX(
3         y,
4         order=order,
5         seasonal_order=seasonal_order,
6         exog=exog,
7         enforce_stationarity=False,
8         enforce_invertibility=False
9     )
10    result = model.fit(dispatch=False)
11    return result

```

8.3 Celda 16: modelo ARIMA base

```

1 y = daily['log_sales'].dropna()
2
3 arima_111 = fit_sarima(y, order=(1,1,1), seasonal_order=(0,0,0,0))
4 print(arima_111.summary())

```

8.4 Celda 17: modelo SARIMA con estacionalidad semanal

Celda Markdown

Como la base es diaria, una primera hipótesis razonable es evaluar estacionalidad semanal con $s = 7$. Si se trabajara con datos mensuales, el periodo natural sería $s = 12$.

```

1 sarima_weekly = fit_sarima(
2     y,
3     order=(1,1,1),

```

```

4     seasonal_order=(1,1,1,7)
5 )
6 print(sarima_weekly.summary())

```

8.5 Celda 18: comparación de modelos

```

1 comparison = pd.DataFrame({
2     'modelo': ['ARIMA(1,1,1)', 'SARIMA(1,1,1)(1,1,1,7)'],
3     'AIC': [arima_111.aic, sarima_weekly.aic],
4     'BIC': [arima_111.bic, sarima_weekly.bic],
5     'LogLik': [arima_111.llf, sarima_weekly.llf]
6 })
7 comparison.sort_values('AIC')

```

9 Modelo SARIMAX con variables exógenas

9.1 Motivación

Celda Markdown

El modelo SARIMAX permite incluir factores externos. En esta base se pueden usar **promo** y **holiday** como variables explicativas. Esto mejora la interpretación porque las ventas no dependen únicamente de su pasado, sino también de decisiones comerciales y efectos de calendario.

9.2 Celda 19: preparación de variables exógenas

```

1 exog = daily[['promo', 'holiday']].copy()
2 exog = exog.loc[y.index]
3
4 sarimax_model = fit_sarima(
5     y,
6     order=(1,1,1),
7     seasonal_order=(1,1,1,7),
8     exog=exog
9 )
10 print(sarimax_model.summary())

```

9.3 Celda 20: comparación final

```

1 comparison = pd.DataFrame({
2     'modelo': [
3         'ARIMA(1,1,1)',
4         'SARIMA(1,1,1)(1,1,1,7)',
5         'SARIMAX + promo + holiday'
6     ],
7     'AIC': [arima_111.aic, sarima_weekly.aic, sarimax_model.aic],
8     'BIC': [arima_111.bic, sarima_weekly.bic, sarimax_model.bic],

```

```

9      'LogLik': [arima_111.llf, sarima_weekly.llf, sarimax_model.llf]
10  })
11  comparison.sort_values('AIC')

```

Interpretación financiera / econométrica

Si el SARIMAX reduce AIC/BIC y las variables exógenas son significativas, entonces el modelo no sólo mejora estadísticamente, sino que también aporta interpretación económica: promociones y feriados contienen información relevante para explicar ventas.

10 Diagnóstico de residuos

10.1 Prueba Ljung–Box

Celda LaTeX / Fórmula

La prueba Ljung–Box evalúa si los residuos presentan autocorrelación conjunta hasta cierto número de rezagos:

$$H_0 : \rho_1 = \rho_2 = \cdots = \rho_m = 0.$$

El estadístico es:

$$Q^* = n(n+2) \sum_{k=1}^m \frac{\hat{\rho}_k^2}{n-k}.$$

Si el p -value es mayor a 0.05, no se rechaza H_0 , lo cual sugiere que los residuos se comportan como ruido blanco.

10.2 Celda 21: diagnóstico de residuos

```

1  resid = sarimax_model.resid.dropna()
2
3  fig, ax = plt.subplots(1, 2, figsize=(14, 4))
4  resid.plot(ax=ax[0])
5  ax[0].set_title('Residuos del modelo SARIMAX')
6  plot_acf(resid, lags=40, ax=ax[1])
7  ax[1].set_title('ACF de residuos')
8  plt.show()
9
10 lb = acorr_ljungbox(resid, lags=[10, 20, 30], return_df=True)
11 lb

```

10.3 Celda 22: normalidad y distribución de residuos

```

1  fig, ax = plt.subplots(1, 2, figsize=(14, 4))
2  ax[0].hist(resid, bins=40, density=True)
3  ax[0].set_title('Histograma de residuos')
4  stats.probplot(resid, dist='norm', plot=ax[1])
5  ax[1].set_title('QQ plot de residuos')

```



```

6 plt.show()
7
8 jb = stats.jarque_bera(resid)
9 print('Jarque-Bera:', jb)

```

Interpretación financiera / econométrica

Un modelo puede capturar adecuadamente la media condicional y aun así tener residuos no normales. En series financieras o comerciales, los outliers y efectos de calendario pueden generar colas pesadas. Por eso el diagnóstico no debe limitarse al AIC/BIC.

11 Pronóstico con intervalos de confianza

11.1 Celda 23: pronóstico fuera de muestra

```

1 steps = 30
2
3 future_index = pd.date_range(
4     start=y.index[-1] + pd.Timedelta(days=1),
5     periods=steps,
6     freq='D'
7 )
8
9 # Escenario base: sin promoción y sin feriado
10 future_exog = pd.DataFrame({
11     'promo': np.zeros(steps),
12     'holiday': np.zeros(steps)
13 }, index=future_index)
14
15 forecast = sarimax_model.get_forecast(steps=steps, exog=future_exog)
16 forecast_mean = forecast.predicted_mean
17 forecast_ci = forecast.conf_int()
18
19 # Regresar de log ventas a ventas
20 forecast_sales = np.exp(forecast_mean)
21 forecast_lower = np.exp(forecast_ci.iloc[:, 0])
22 forecast_upper = np.exp(forecast_ci.iloc[:, 1])

```

11.2 Celda 24: gráfica de pronóstico

```

1 fig, ax = plt.subplots(figsize=(12, 5))
2 np.exp(y).iloc[-120:].plot(ax=ax, label='Histórico')
3 forecast_sales.plot(ax=ax, label='Pronóstico')
4 ax.fill_between(
5     future_index,
6     forecast_lower,
7     forecast_upper,
8     alpha=0.2,
9     label='Intervalo de confianza'
10 )

```

```

11 ax.set_title('Pronóstico SARIMAX de ventas')
12 ax.set_ylabel('Ventas')
13 ax.legend()
14 plt.show()

```

Interpretación financiera / econométrica

El pronóstico debe leerse como un escenario condicional. Si las variables futuras **promo** y **holiday** cambian, el pronóstico también cambia. Por eso, en un negocio real conviene construir varios escenarios: base, campaña promocional y calendario con feriados.

12 Apéndice teórico: ejercicios SARIMAX y GARCH

12.1 SARIMAX con variable exógena

Celda Markdown

En los ejercicios analíticos del proyecto se trabaja un modelo SARIMAX con variable exógena macroeconómica. La idea central es que una serie puede depender tanto de su propia dinámica temporal como de un regresor externo x_t .

Celda LaTeX / Fórmula

Un ejemplo documentado es:

$$SARIMAX(1, 1, 1)(1, 1, 0)_{12}.$$

Con variable exógena x_t , puede escribirse como:

$$(1 - \Phi_1 B^{12})(1 - \phi_1 B)(1 - B)(1 - B^{12})y_t = \beta x_t + (1 + \theta_1 B)\varepsilon_t.$$

12.2 Prueba ARCH

Celda Markdown

La prueba ARCH se usa para detectar heterocedasticidad condicional. Es especialmente importante en series financieras, donde puede no haber autocorrelación significativa en los rendimientos, pero sí autocorrelación en los rendimientos al cuadrado.

Celda LaTeX / Fórmula

La hipótesis nula de la prueba LM-ARCH es:

$$H_0 : \alpha_1 = \alpha_2 = \dots = \alpha_m = 0.$$

Si se rechaza H_0 , existe evidencia de efecto ARCH y puede ser adecuado estimar un modelo GARCH.

12.3 Modelo GARCH(1,1)

Celda LaTeX / Fórmula

El modelo GARCH(1,1) se define como:

$$r_t = \mu + \varepsilon_t, \quad \varepsilon_t = \sigma_t z_t, \quad z_t \sim N(0, 1),$$

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2.$$

La condición de estacionariedad en covarianza es:

$$\alpha + \beta < 1.$$

La varianza incondicional es:

$$\bar{\sigma}^2 = \frac{\omega}{1 - \alpha - \beta}.$$

Interpretación financiera / econométrica

Cuando $\alpha + \beta$ está cerca de uno, la volatilidad es muy persistente. Esto significa que un choque fuerte puede tardar muchos periodos en disiparse, generando clustering de volatilidad.

12.4 GJR-GARCH y efecto apalancamiento

Celda LaTeX / Fórmula

El modelo GJR-GARCH incorpora asimetría:

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \lambda \varepsilon_{t-1}^2 \mathbf{1}_{\{\varepsilon_{t-1} < 0\}} + \beta \sigma_{t-1}^2.$$

Si $\lambda > 0$, los choques negativos incrementan más la volatilidad futura que choques positivos de la misma magnitud.

12.5 EGARCH

Celda LaTeX / Fórmula

El modelo EGARCH modela el logaritmo de la varianza:

$$\log(\sigma_t^2) = \omega + \beta \log(\sigma_{t-1}^2) + \alpha (|z_{t-1}| - E|z_{t-1}|) + \gamma z_{t-1}.$$

Su ventaja es que garantiza positividad de la varianza sin imponer restricciones de signo sobre los parámetros.

12.6 GARCH-M

Celda LaTeX / Fórmula

En GARCH-M, la volatilidad entra en la ecuación de la media:

$$r_t = \mu + \delta \sigma_t + \varepsilon_t.$$

El parámetro δ mide una prima de riesgo dependiente de la volatilidad condicional.

Interpretación financiera / econométrica

Si $\delta > 0$, una mayor volatilidad esperada se asocia con un mayor rendimiento esperado. Esto es consistente con la intuición financiera de que los inversionistas exigen compensación por asumir más riesgo.

13 Celda opcional: estimación GARCH en Python

Celda Markdown

Esta sección es opcional y aplica cuando la serie que se modela son rendimientos financieros. Para ventas, normalmente primero se modela la media con SARIMA/SARIMAX. Para activos financieros, después de modelar la media puede estimarse GARCH sobre residuos o rendimientos.

```

1 # Ejemplo opcional si se tienen rendimientos financieros:
2 # returns = 100 * daily['diff_log_sales'].dropna()
3
4 returns = 100 * daily['diff_log_sales'].dropna()
5
6 if arch_model is not None:
7     garch = arch_model(returns, mean='Constant', vol='GARCH', p=1, q=1, dist=
8         'normal')
9     garch_res = garch.fit(dispatch='off')
10    print(garch_res.summary())
11 else:
12    print('La librería arch no está disponible. Ejecuta: !pip install arch')
```

Cuidado metodológico

Aunque el código permite estimar un GARCH sobre diferencias logarítmicas de ventas, la interpretación debe hacerse con cuidado. GARCH es más natural en rendimientos financieros porque modela volatilidad condicional; en ventas puede ser útil sólo si el objetivo es analizar variabilidad temporal de crecimiento o errores.

14 Conclusiones del notebook

1. El notebook queda separado correctamente como material de **Series de Tiempo**; no incorpora temas de Cálculo Estocástico ni valuación de derivados.
2. La base `store_sales.csv` permite construir una serie diaria de ventas y estimar modelos ARIMA, SARIMA y SARIMAX.
3. ARIMA modela la dinámica regular; SARIMA añade estacionalidad; SARIMAX permite incorporar variables exógenas como promoción y feriados.
4. La estacionariedad se revisa mediante ADF, gráficas, transformación logarítmica y diferenciación.
5. ACF y PACF sirven para proponer parámetros iniciales, pero la selección final debe validar par-simonia, AIC/BIC, significancia y residuos.
6. La prueba Ljung–Box permite verificar si los residuos se comportan como ruido blanco.
7. Los modelos ARCH/GARCH complementan el análisis cuando el interés está en volatilidad condicional, especialmente en series financieras.

Interpretación financiera / econométrica

El resultado final es un guion de Colab que no sólo corre modelos, sino que explica qué se está haciendo, por qué se hace y cómo interpretar los resultados. Esto convierte el análisis en una entrega académica defendible, reproducible y bien documentada.

15 Checklist final para ejecutar en Colab

Cuadro 5: Checklist de revisión del notebook

No.	Actividad	Estado
1	Cargar <code>store_sales (1).csv</code> .	☐
2	Verificar columnas, tipos de datos y valores faltantes.	☐
3	Convertir <code>date</code> a formato fecha y ordenar la base.	☐
4	Agregar ventas por fecha y definir frecuencia diaria.	☐
5	Graficar ventas diarias, ventas mensuales y patrón semanal.	☐
6	Aplicar transformación logarítmica y diferenciación si es necesario.	☐
7	Ejecutar prueba ADF en serie original y transformada.	☐
8	Revisar ACF y PACF.	☐
9	Estimar ARIMA base.	☐
10	Estimar SARIMA con estacionalidad semanal.	☐
11	Estimar SARIMAX con <code>promo</code> y <code>holiday</code> .	☐
12	Comparar AIC, BIC y log-verosimilitud.	☐
13	Diagnosticar residuos con Ljung–Box, histograma y QQ plot.	☐
14	Generar pronóstico con intervalos de confianza.	☐
15	Escribir interpretación financiera y conclusión.	☐

Referencias

- [1] Box, G. E. P., Jenkins, G. M., Reinsel, G. C. y Ljung, G. M. *Time Series Analysis: Forecasting and Control*. Wiley.
- [2] Hamilton, J. D. *Time Series Analysis*. Princeton University Press.
- [3] Hyndman, R. J. y Athanasopoulos, G. *Forecasting: Principles and Practice*. OTexts.
- [4] Engle, R. F. (1982). Autoregressive Conditional Heteroskedasticity with Estimates of the Variance of United Kingdom Inflation. *Econometrica*.
- [5] Bollerslev, T. (1986). Generalized Autoregressive Conditional Heteroskedasticity. *Journal of Econometrics*.
- [6] Nelson, D. B. (1991). Conditional Heteroskedasticity in Asset Returns: A New Approach. *Econometrica*.
- [7] Material del curso de Series de Tiempo proporcionado en los archivos `Reporte Sarima`, `Proyecto ST.Escamilla`, `Guevara`, `Proyecto Series`, `README.md`, `store_sales.csv` e `informe_arima_sarima_aapl`.